

SMART CAMPUS RESOURCE MANAGEMENT SYSTEM

Project Report - Build Your Own Project (VITyarthi)

Submitted by: Krishnabhishek Rana

Course: Java Programming / Object-Oriented Programming

Date: 17 September 2026

1. Introduction

Campus rooms, laboratories, and shared equipment are frequently requested by many students. Manual or informal booking causes duplicate reservations and gives administrators poor visibility into demand. Smart Campus is a Java console application that stores students and resources, accepts a reservation, and reports resource use.

2. Problem Statement and Objectives

The project prevents duplicate resource allocation while providing a clear workflow for campus administrators. Its objectives are student management, resource management, booking management, basic analytics, and a safe simulation of concurrent booking requests.

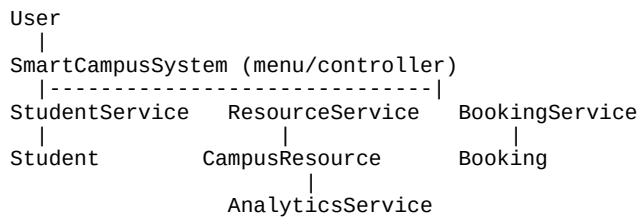
3. Functional Requirements

1. Add, list, search, and sort students. 2. Add and display campus resources. 3. Create and view bookings. 4. Reject invalid input, unknown IDs, and duplicate bookings. 5. Show analytics. 6. Demonstrate concurrent booking safety.

4. Non-Functional Requirements

Usability: numbered interactive menu and readable output. Reliability: validation and descriptive errors. Performance: HashMap lookups. Maintainability: responsibility is separated among model, service, and utility classes. Concurrency safety: a synchronized booking method blocks race conditions. Resource efficiency: the program uses in-memory storage and no external dependencies.

5. System Architecture



6. Workflow Diagram

Start -> Display menu -> Select feature -> Validate input
-> Student/Resource/Booking service -> Display result -> Menu
Booking path: check student -> check resource -> synchronized reserve -> record booking

7. Use Case and Class Design

```
Actor: Administrator / Student
|-- Manage Students
|-- Manage Resources
|-- Book Resource
|-- View Bookings and Analytics
|-- Run Concurrent Booking Demo

StudentService -> Student
ResourceService -> CampusResource
BookingService -> Booking, ResourceService
AnalyticsService -> StudentService, ResourceService, BookingService
```

8. Sequence Diagram: Booking

```
User -> Menu: enter student/resource/purpose
Menu -> StudentService: findById()
Menu -> BookingService: create()
BookingService -> ResourceService: findById()
BookingService -> CampusResource: isAvailable(), reserve()
BookingService -> Booking: create record
BookingService -> User: confirmation or error
```

9. Design Decisions and Implementation

The application is split into ten meaningful Java classes. Student, CampusResource, and Booking represent the domain data. Service classes isolate storage and business logic. InputValidator avoids repeated validation code. BookingAttempt implements Runnable for the concurrency demonstration. HashMap supports fast retrieval by ID, ArrayList retains booking order, and synchronization ensures only one thread can reserve a resource at a time.

10. Results and Testing

The program was compiled with javac and tested with the sample student list, a valid booking, analytics display, and two simultaneous attempts to reserve the Innovation Lab. The first request succeeded and the second was rejected as already booked. Additional validation tests include blank input, duplicate IDs, unknown student IDs, and unknown resource IDs.

11. Challenges, Learnings, and Future Work

The main challenge was protecting a shared resource from concurrent requests. This project demonstrates encapsulation, collections, exception handling, Runnable, Thread, synchronized methods, and modular design. Future improvements include persistent database storage, login roles, booking cancellation, time slots, a GUI or web interface, and email notifications.

12. References

Java Platform documentation, Oracle. Course concepts and VITyarthi project guidelines supplied by the institution.